0X00

```
int __fastcall main(int argc, const char **argv, const char **envp)
{
  unsigned int v4[5]; // [rsp+4h] [rbp-14h] BYREF

  v4[0] = 0;
  v4[1] = 0;
  v4[2] = 0;
  while ( *((_DWORD *)get_cell(v4) + 3) != 3 )
  {
    putchar(10);
    prompt_and_update_pos(v4);
  }
  puts("You break into the vault and read the secrets within...");
  get_flag("You break into the vault and read the secrets within...", argv);
  return 0;
}
```

```
char *__fastcall get_cell(unsigned int *a1)
{
  return (char *)&maze + 6400 * *a1 + 320 * a1[1] + 16 * a1[2];
}
```

```
Direction (L/R/F/B/U/D/Q)?
Direction (L/R/F/B/U/D/Q)?
```

可以知道，这个应该是要输入一个 L/R/F/B/U/D/Q 的序列，然后就可以 hack in 了。

```
.rodata:00000000000020E0                 public maze
.rodata:00000000000020E0 maze            db    0
.rodata:00000000000020E1                 db    0
.rodata:00000000000020E2                 db    0
.rodata:00000000000020E3                 db    0
.rodata:00000000000020E4                 db    0
.rodata:00000000000020E5                 db    0
.rodata:00000000000020E6                 db    0
.rodata:00000000000020E7                 db    0
.rodata:00000000000020E8                 db    0
.rodata:00000000000020E9                 db    0
.rodata:00000000000020EA                 db    0
.rodata:00000000000020EB                 db    0
```

可以看到 maze 是一个只读的结构体，我们应该是找到其中为 3 的值，然后就可以跳出 while 循环了。

然后从 get_cell 函数可以知道，这是一个 3 dim 坐标映射到 1 元坐标系的迷宫题。

让我们看看，哪些地方可以走，哪些地方不可以走吧。

```
switch ( v8[0] )
{
  case 'B':
    v2 = *((_DWORD *)a1 + 1);
    if ( !v2 )
      goto LABEL_32;
    HIDWORD(v6) = v2 - 1;
    if ( *((_DWORD *)get_cell((unsigned int *)&v6) + 3) == 2 )
      goto LABEL_33;
    *a1 = v6;
    result = v7;
    *((_DWORD *)a1 + 2) = v7;
    break;
```

```
LABEL_32:
        result = puts("Cannot move that way");
      }
      else
      {
        v7 = v5 + 1;
        if ( *((_DWORD *)get_cell((unsigned int *)&v6) + 3) == 2 )
        {
LABEL_33:
          result = puts("Cannot move that way");
        }
        else
        {
          *a1 = v6;
          result = v7;
          *((_DWORD *)a1 + 2) = v7;
        }
```

这里以 B 为例，可以知道 label 32 和 label 33 是错误处理的分支，我们要让他不走到这里的情况是，往 b 走一步再+12 个字节，不能碰到 2。可以看到 b 是 Y-1。

同理可得

R >>> X+1

L >>> X-1

U >>> Z+1

D >>> Z-1

F >>> Y+1

B >>> Y-1

它们共用一个条件，不能碰到 2。

0X10

答案已经呼之欲出，我们用 BFS，广度优先遍历，找到 3，就可以了。

IDA pro 脚本：

```
import idc
import idaapi
from collections import deque

# --- 核心配置 ---
MAZE_BASE = 0x20E0
DIM = 20
STATUS_OFFSET = 12  # 指针 +3 对应字节偏移 +12

def get_cell_addr(x, y, z):
    # 严格遵循源码: maze[800 * X + 40 * Y + 2 * Z]
    return MAZE_BASE + (6400 * x + 320 * y + 16 * z)

def get_status(x, y, z):
    """统一获取状态值: 读取偏移 12 字节处的 DWORD"""
    if not (0 <= x < DIM and 0 <= y < DIM and 0 <= z < DIM):
        return 2  # 越界视为墙

    addr = get_cell_addr(x, y, z)
    # 模拟 *((_DWORD *)get_cell(...) + 3)
    return idc.get_wide_dword(addr + STATUS_OFFSET)
```

```python
def solve():
    print("[*] 正在启动 BFS 寻路算法（统一使用 +12 偏移检测）...")

    # 初始起点
    start_pos = (0, 0, 0)
    queue = deque([(start_pos, "")])
    visited = {start_pos}

    # 移动方向
    moves = [
        ('L', (-1, 0, 0)), ('R', (1, 0, 0)),
        ('F', (0, 1, 0)),  ('B', (0, -1, 0)),
        ('U', (0, 0, 1)),  ('D', (0, 0, -1))
    ]

    while queue:
        (cx, cy, cz), path = queue.popleft()

        # 1. 检查是否已经是终点
        if get_status(cx, cy, cz) == 3:
            print("-" * 40)
            print(f"[+] 路径找到！")
            print(f"[+] 终点坐标: ({cx}, {cy}, {cz})")
            print(f"[+] 最终路径: {path}")
            print("-" * 40)
            return path

        # 2. 探索下一步
        for direction, (dx, dy, dz) in moves:
            nx, ny, nz = cx + dx, cy + dy, cz + dz

            if (nx, ny, nz) not in visited:
                status = get_status(nx, ny, nz)

                # 只有状态不为 2 (墙) 时才移动
                if status != 2:
                    visited.add((nx, ny, nz))
                    queue.append(((nx, ny, nz), path + direction))

    print("[-] 搜索结束，未找到通往 3 的路径。")
```

然后就可以得到:

[+] 路径找到!

[+] 终点坐标: (19, 19, 19)

[+] 最终路径:
UUURFURURRFRRFFUUFURRUFUFFRFUFUUUUFFRRUUUFURFDFFUFFRRRRRRFRR

☹️不过我很无语，我的 ida 最开始给我分析错了，正确的步长是 16，但是最开始给我

分析成 2 了…无语。